Integrating Neural Amp Modeling into Csound

Author

InstituteA
email@institutedomain.com

Abstract. Neural Amp Modeler (NAM) is the most widely used open-source system for the black-box modeling of nonlinear audio hardware using neural networks. A community library of several hundred thousand pre-trained captures exists, but has yet to be integrated into the Csound ecosystem. This paper presents `NAMProcess`, a Csound opcode that performs real-time NAM inference. The opcode loads standard `.nam` model files, switches between models without interrupting audio, adds no algorithmic latency, and runs comfortably in real time on systems like the Raspberry Pi. A live dry-referenced volume compensation system built into the opcode holds every capture in a 25-model test library within ±2 dB of the player's volume. This paper details the implementation of a thin wrapper around the NeuralAmpModelerCore C++ library and its integration as a native plugin opcode, a drag-and-drop Cabbage plugin, and "tone" section of a Raspberry Pi-based guitar pedal.

Keywords: Neural Amp Modeling, Raspberry Pi, Cabbage Plugin, Opcode, Volume Compensation

1 Introduction

Over the past several years a new approach has become standard practice in guitar audio: record a real device, then train a neural network to reproduce its input–output behavior. Neural Amp Modeler (NAM) [2,3] is the most widely used open-source implementation of this approach. Its model format has become standard with over 350,000 captures accumulated on TONE3000 alone [16] covering a variety of non-linear hardware including amplifiers, effect pedals and preamps, freely downloadable.

2 Background and Related Work

Feedforward WaveNet variants and recurrent LSTM networks are the established black-box models of guitar amplifiers and distortion circuits, and both run in real time [7,8,19,20,21]. NAM [2,3] packages both architectures behind a single model format and a training pipeline whose input is a fixed published reference signal reamped through the target device.

Real-time inference of such models already exists in the other major environments. RTNeural [6] is a general inference library for audio callbacks, with a SuperCollider UGen and Max/MSP and Pure Data ports [14]. The `neural~` external loads `.nam` files in Max/MSP [15], `neural-amp-modeler-lv2` embeds the NAM core in an LV2 plugin [13], and anira treats real-time-safe inference in audio callbacks generally [1]. Csound [12] has no equivalent. Its prior guitar-gear modeling derives from circuit knowledge rather than data, like the component-level triode stage opcode of Fink and Rabenstein [9] or Rory Walsh's model of the Ibanez Tube Screamer pedal [18]. `NAMProcess` closes that gap.

3 The NAMProcess Opcode

3.1 Interface

```
aoutL, aoutR NAMProcess ainL, ainR,
      kProfileIdx, SLibDir
      [, kAutoVol]
icount       NAMCount
```

`SLibDir` points at a folder, scanned recursively for `.nam` files at init. The sorted file list gives every model a stable index, and `kProfileIdx` picks one at k-rate. Writing a new index kicks off an asynchronous load, and the running model keeps playing until the new one is ready. `NAMCount` reports the library size so orchestra logic can wrap or clamp. Since model choice is just another k-rate value, modulating high-quality NAM models is now open to the same rule-based and generative processes Koenig practiced [10]. `kAutoVol` enables the volume compensation of Section 4, on by default. A minimal live instrument follows.

```
instr 1
  aL, aR  ins
  kIdx    chnget "model" ; MIDI, OSC, score
  aoL,aoR NAMProcess aL, aR, kIdx,
          "/home/user/models"
  outs aoL, aoR
endin
```

Until the first model loads, or if the folder holds no models, audio passes through untouched so a misconfigured path degrades to bypass instead of silence.

3.2 Loader and Model State

Everything that touches files or network weights runs on a background thread owned by the opcode [5]. The audio thread never allocates, parses JSON, or blocks on I/O. Per k-cycle it takes one uncontended mutex acquisition to copy a pointer to the active model pair. The loader builds the requested model, picks the full-quality submodel (for when "slimmable models" are an option) [2], and pre-warms internal buffers before publishing the swap. A failed load keeps the previous model running, so a corrupt file cannot interrupt a performance, and swaps land on a block boundary to keep audio free of clicks.

A model trained at a different sample rate than the engine gets wrapped in a Lanczos resampling container so the network always runs at its trained rate. A 48 kHz model under a 44.1 kHz engine matches the matched-rate reference within 0.3 dB.

NAM models are stateful. WaveNet layers keep convolution history in ring buffers and LSTMs carry hidden state between calls. So `NAMProcess` loads two independent instances of the same file, resets both off-thread, and publishes them as a pair, one per channel. Running both channels through one shared instance convolves every block against the other channel's immediate past, which measures as -4.7 dB crosstalk on decorrelated stereo material.

3.3 Latency

The supported architectures are causal, so the opcode itself should add no delay. The latency at 128 samples is 2.7 ms at 48 kHz in the pedal deployment of Section 6.

4 Live Volume Compensation

The opcode continuously matches the wet output to the perceived loudness of the player's dry signal, so loading any capture (clean, cranked, mislabeled) leaves the volume unchanged.

Without this, browsing a community library is impractical live. Fed the same signal, the 25 representative captures of Fig. 2 span 17 dB even after a load-time normalization probe, and embedded loudness metadata does not predict the ranking. No pre-computed correction can close the gap, because saturating models compress: one compression-dominated capture holds its output within 1 dB across a 14 dB input sweep, so no per-model gain equalizes it at all. The correction has to be relative to what the player is actually doing live.

Dry input and wet output are metered as ITU-R BS.1770 K-weighted mean squares averaged with a 2.5 s time constant, and a slow output trim tracks their ratio (Fig. 1). Neither meter observes the trim's own effect, so the control loop is feed-forward and cannot oscillate. Program dynamics move both meters together and leave the ratio untouched to ensure the mechanism does not add any feeling of compression. The goal is a system where loudness is held constant while spectrum and saturation

can vary. Measured after convergence, a −12 dB input step passes through as −10.2 dB of output, the residual being the amp's own compression. The trim engages after one second of active signal, slews at most 2 dB/s, holds within a 0.75 dB deadband, freezes below −55 dBFS input, and is clamped to ±18 dB. Converged trims are remembered per model file, so returning to a played capture lands at level immediately (+0.6 dB measured).

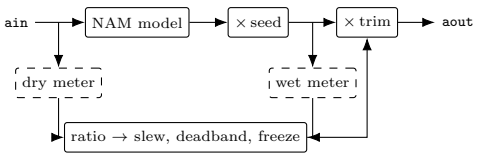

Fig. 1. Live compensation topology inside one `NAMProcess` cycle. The dry meter taps the raw input and the wet meter taps after the model, before the trim.

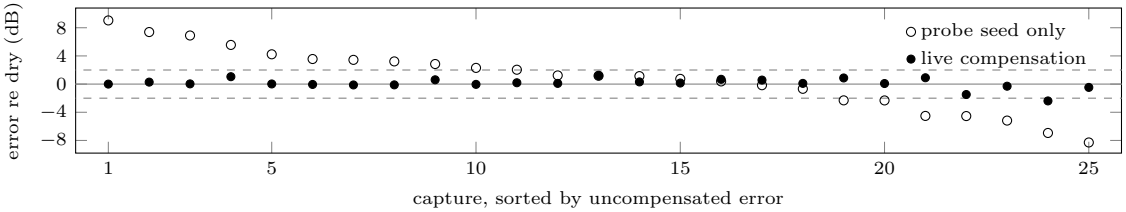

Fig. 2. Loudness error relative to the dry input, all 25 captures on identical material (saw plucks, peak 300 ms RMS, settled), sorted by uncompensated error. Dashed lines mark ±2 dB.

Figure 2 measures the shipped system: the live loop collapses the 17.3 dB seed-only spread (standard deviation 4.3 dB) to 3.6 dB (0.74 dB), all but one capture within ±2 dB of the dry signal, including the compression-dominated capture that no input gain can equalize. On the guitar material used during development the corresponding spread is 3.1 dB (0.83 dB), every capture inside ±2 dB. The best purely static probe evaluated left 9.9 dB (2.60 dB). The official NAM plugin's normalized output mode [4] applies a static gain from stored metadata or a fixed target. To my knowledge, no published NAM host performs live dry-referenced compensation.

5 A Drag-and-Drop Cabbage Plugin

This opcode is accessible as a VST3/AU plugin through Cabbage [17]. Cabbage publishes the dropped file's path on its `LAST_FILE_DROPPED` channel, and a path-input loader variant hot-swaps to it through the background mechanism of Section 3.2. The UI reports the loaded model, but provides arrows to navigate other NAM files in the same parent folder. The compensation of Section 4 defaults on, but can be toggled off exposing raw model level. The input and output knobs are clean gains in dB. The input gain feeds both the model and the dry reference of the compensation, so it acts as plain volume while the model supplies the saturation. A gate, three-band tone stack, and DC blocker mirror the outboard chain of the official NAM plugin [4]. An orchestra that fails to compile emits silence yet passes host validation, so the base orchestra contains no NAM references and the engine instrument is compiled at runtime with `compilestr`. If the opcode library fails to load, audio passes through dry and the failure is reported on the status display.

6 Raspberry Pi Guitar Pedal

The opcode's original deployment target is a Raspberry Pi 5-based guitar pedal running Csound 6.18 at 48 kHz with `ksmps` 128. The audio chain pinned to one dedicated core, where the 25-capture library

acts as a bank of "tones". Model selection is mapped to MIDI foot controls. Loading is click-safe and volume matched so the player can smoothly switch between amps mid-performance. The only added latency is the block granularity of Section 3.3 (2.7 ms), well inside a live monitoring budget. CPU load with only a full-quality NAM model running has been informally observed around 15% of the audio core during performance.

7 Conclusions

`NAMProcess` gives Csound direct access to NAM's model format and community library as a drop-in nonlinear processor addressable by a k-rate index, shipping as a Cabbage plugin and a Raspberry Pi implementation. Source code and measurement tooling will be released under an open-source license upon publication.

References

1. Ackva, V., Schulz, F.: anira: An Architecture for Neural Network Inference in Real-Time Audio Applications. In: Proceedings of the 5th IEEE International Symposium on the Internet of Sounds (IS2) (2024)
2. Atkinson, S.: Slimmable NAM: Neural Amp Models with Adjustable Runtime Computational Cost. arXiv:2511.07470 (2025)
3. Atkinson, S.: Neural Amp Modeler and NeuralAmpModelerCore, `https://www.neuralampmodeler.com`, `https://github.com/sdatkinson/NeuralAmpModelerCore`
4. Atkinson, S.: Neural Amp Modeler Plugin, `https://github.com/sdatkinson/NeuralAmpModelerPlugin`
5. Bencina, R.: Real-time Audio Programming 101: Time Waits for Nothing, `http://www.rossbencina.com/code/real-time-audio-programming-101-time-waits-for-nothing`
6. Chowdhury, J.: RTNeural: Fast Neural Inferencing for Real-Time Systems. arXiv:2106.03037 (2021)
7. Damskägg, E.-P., Juvela, L., Thuillier, E., Välimäki, V.: Deep Learning for Tube Amplifier Emulation. In: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 471–475. Brighton (2019)
8. Damskägg, E.-P., Juvela, L., Välimäki, V.: Real-Time Modeling of Audio Distortion Circuits with Deep Learning. In: Proceedings of the 16th Sound and Music Computing Conference (SMC), pp. 332–339. Málaga (2019)
9. Fink, M., Rabenstein, R.: A Csound Opcode for a Triode Stage of a Vacuum Tube Amplifier. In: Proceedings of the 14th International Conference on Digital Audio Effects (DAFx-11), pp. 365–370. Paris (2011)
10. Koenig, G.M.: Project 2: A Programme for Musical Composition. Electronic Music Reports 3. Institute of Sonology, Utrecht State University, Utrecht (1970)
11. Lazzarini, V.: The Csound Plugin Opcode Framework. In: Proceedings of the 14th Sound and Music Computing Conference (SMC), pp. 267–274. Espoo (2017)
12. Lazzarini, V., Yi, S., ffitch, J., Heintz, J., Brandtsegg, Ø., McCurdy, I.: Csound: A Sound and Music Computing System. Springer, Cham (2016)
13. Oliphant, M.: neural-amp-modeler-lv2, `https://github.com/mikeoliphant/neural-amp-modeler-lv2`
14. Pluta, S.: RTNeural Real-Time Neural Inference UGen, `https://scsynth.org/t/rtneural-real-time-neural-inference-ugen/10074`
15. Presta, A.: neural_tilde: Neural Amp Modeler External for Max/MSP, `https://github.com/apresta/neural_tilde`
16. TONE3000, `https://www.tone3000.com`
17. Walsh, R.: Developing Csound Plugins with Cabbage. In: J. Heintz, A. Hofmann, I. McCurdy (eds.) Ways Ahead: Proceedings of the First International Csound Conference. Cambridge Scholars Publishing, Newcastle upon Tyne (2013)
18. Walsh, R., Walsh, C.: Digital Signal Processing Techniques Used to Model the Ibanez Tube Screamer Guitar Pedal. In: Proceedings of the 5th International Csound Conference (ICSC 2019). Cagli (2019)
19. Wright, A., Damskägg, E.-P., Välimäki, V.: Real-Time Black-Box Modelling with Recurrent Neural Networks. In: Proceedings of the 22nd International Conference on Digital Audio Effects (DAFx-19), pp. 173–180. Birmingham (2019)
20. Wright, A., Damskägg, E.-P., Juvela, L., Välimäki, V.: Real-Time Guitar Amplifier Emulation with Deep Learning. Applied Sciences 10(3), 766 (2020)
21. Wright, A., Välimäki, V.: Perceptual Loss Function for Neural Modelling of Audio Systems. In: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), pp. 251–255. Barcelona (2020)